



SMART INDIA
HACKATHON
2026

PROJECT REPORT

iTantra

Indian Multilingual TTS & STT Aided Neural Transceiver

Radio Access for Low Bitrate Links

Speech goes in one end. Speech comes out the other. In between it becomes a few dozen bytes.

Field	Value
Problem Statement ID	26173
Problem Statement Title	iTantra — Indian Multilingual TTS & STT Aided Neural Transceiver Radio Access for low bitrate links
Organisation	Indian Space Research Organisation (ISRO)
Department	Department of Space
Theme	Smart Automation
PS Category	Software
Team Name	Taraket
Team ID	—
Report date	22 September 2026
Repository licence	Apache-2.0

SUBMITTED BY

Team Taraket

Smart India Hackathon 2026 · Problem Statement 26173 · Smart Automation
Indian Space Research Organisation, Department of Space

Contents

Abstract

1. The problem

- 1.1 What the problem statement asks for
- 1.2 Why a codec cannot solve it

2. Proposed solution

- 2.1 The idea
- 2.2 Cross-language delivery falls out of the design
- 2.3 What is novel

3. System architecture

- 3.1 The signal path

4. Wire protocol and compression

- 4.1 The frame
- 4.2 Three tiers, chosen per utterance
- 4.3 Measured compression

5. Radio transports and the mesh

6. Security

7. Implementation

8. How the numbers are measured

- 8.1 Not with a stopwatch
- 8.2 Two end-to-end figures, and why
- 8.3 What makes a run reportable

9. Measured results

- 9.1 Where the time goes
- 9.2 End to end
- 9.3 Link characterisation
- 9.4 A figure we are not quoting
- 9.5 Footprint

10. Feasibility and viability

11. Impact and benefits

- 11.1 Who it is for
- 11.2 What changes
- 11.3 Beyond the problem statement

12. Limitations and what remains

13. References

- 13.1 Models, runtimes and datasets
- 13.2 Standards and specifications
- 13.3 Competition
- 13.4 This project

Abstract

Two people hold two ordinary Android phones. One speaks Hindi; the other hears Hindi a second later. There is no SIM in either handset, no cell tower within reach and no server anywhere in the path — the whole exchange runs in aeroplane mode over Bluetooth.

What crosses the radio link is not audio. iTantra recognises the speech on the sending phone, sends the recognised sentence as a few dozen encrypted bytes, and synthesises it again on the receiving phone. Both people only ever speak and listen; the text in between is machinery they never see.

That single decision is what makes the problem tractable. A voice codec compresses the waveform, and a waveform detailed enough to be understood has an irreducible size — about 6 kbps even with Opus. Meaning has no such floor. Over 36 measured utterances the average frame on the wire was 34 bytes, against 96,000 bytes of raw PCM for the same three seconds of speech.

This report describes the system, the protocol, the measurement method, and what the measurements actually say — including the places where they fall short of the budget and why. Every figure quoted here was produced by the shipped application on two retail handsets and exported from the device as CSV. None of them are estimates.

1. The problem

1.1 What the problem statement asks for

Problem Statement 26173 asks for a multilingual speech transceiver that works over low-bitrate links: the kind of link available to a disaster-response team, a border patrol or a remote field station, where a voice call is not an option because the bandwidth does not exist. The requirement is speech in and speech out, across Indian languages, with the delay between the two small enough that people can hold a conversation.

1.2 Why a codec cannot solve it

The instinct is to reach for a better audio codec. The arithmetic says no. Three seconds of intelligible speech, sampled at 16 kHz in 16-bit mono, is 96 kB. Opus at its practical floor of 6 kbps brings that to roughly 2.25 kB. On a 300 bps link — the sort of budget an HF or long-range LoRa channel offers — that is still a minute of transmission for three seconds of speech.

The reason is structural. A codec preserves the waveform, and a waveform carries a great deal that a listener does not need: the exact timbre of the speaker, the room, the microphone. Strip all of that and what remains is a sentence. A sentence in an Indian language is a few dozen bytes.

Representation	3-second sentence	Ratio	Fits 300 bps?
Raw PCM, 16 kHz 16-bit mono	96,000 B	1×	No — 43 minutes
Opus at 6 kbps (practical floor)	2,250 B	43×	No — 60 seconds
iTantra, packed and encrypted	52 B	1,846×	Yes — 1.4 s
iTantra template code, encrypted	21 B	4,571×	Yes — 0.6 s

We are not transmitting audio. We are transmitting meaning — and meaning is small.

2. Proposed solution

2.1 The idea

The sending handset runs an offline speech recogniser. The recognised sentence is packed, sealed and put on the radio. The receiving handset unpacks it, and an offline voice reads it aloud. Neither operator ever sees text unless they want to; the interface is a push-to-talk button and a speaker.

Every model runs on the device. There is no network call at runtime, which is not a privacy flourish — it is the requirement. A system that needs a server to translate speech is useless in exactly the situation this problem statement describes.

2.2 Cross-language delivery falls out of the design

Because what travels is a sentence rather than a sound, the receiving handset can synthesise it in whichever voice it has. For the fixed alert vocabulary this goes further: an alert is sent as a one-byte template code, and the receiving phone renders that code from its own table, in its own language. A Tamil speaker sending an evacuation order reaches a Hindi speaker in Hindi, with no translation model anywhere in the system, because the two handsets share a numbered table rather than a string.

This is the most interesting property the architecture has, and it costs one byte on the wire.

2.3 What is novel

- Semantic transport rather than waveform transport, which is what makes a sub-kilobit link usable for speech at all.
- Template codes that are language-independent by construction, giving cross-language alerts without translation.
- A three-tier compression ladder — template code, script-packed text, raw UTF-8 — chosen per utterance, so the common case is cheap and the rare case still works.
- Measurement built into the shipped application: the product exports its own evidence, and refuses to label it reportable when the conditions were not met.

3. System architecture

The application is a Kotlin multi-module Android project. Eight modules, 132 source files and 79 test files. The separation is not cosmetic — the wire format, the recogniser and the radio each have to be testable without the other two, and most of the test suite runs on a plain JVM with no device attached.

Module	Responsibility	Contents
app	Interface and orchestration	Compose screens, push-to-talk control, roster, settings, instrumentation strip, foreground service
core-audio	Capture and playback	AudioRecord/AudioTrack wrappers, ring buffers, gain, alert audio policy, focus handling
core-asr	Speech to text	sherpa-onnx binding, endpointing, hotword and gazetteer biasing, confidence scoring
core-tts	Text to speech	Per-language normalisation, phrase shaping, chunked synthesis, playout scheduling
core-link	Transport	One Link interface over RFCOMM, BLE broadcast, Wi-Fi and serial; discovery, reconnection, air buffer
core-proto	Wire format	Frame encode/decode, CRC-16, AES-GCM, script packing, template tables, replay window, relay logic
core-models	Asset management	Pack manifest, checksum verification, resumable download, storage accounting
bench	Evaluation	WER harness, latency logger and stage decomposition, noise mixing, CSV and scorecard export

3.1 The signal path

One utterance, end to end, touches every module in order:

```

press → capture → recognise → pack → seal → radio
↓ speaker ← synthesise ← normalise ← open ← de-frame ← radio

```

The push-to-talk control sends on release rather than on press, mirroring radio practice: the message is what was said while the control was held, so it goes when the operator lets go. The operator, not an endpointing algorithm, decides where the sentence ends.

4. Wire protocol and compression

4.1 The frame

Every frame carries a ten-byte header — magic and version, type and language, sequence, flags, length, source node, key identifier and a hop count — followed by the payload and a CRC-16 trailer. The header is deliberately small because on a 300 bps link the header is not free.

4.2 Three tiers, chosen per utterance

The sender picks the cheapest representation that carries the sentence:

- Template code — one byte, when the sentence matches the shared alert table. This is the tier that gives cross-language delivery.
- Script-packed text — Indic scripts occupy a contiguous Unicode block, so a codepoint in a declared script fits in one byte instead of the three UTF-8 would spend on it.

- Raw UTF-8 — the fallback, when neither of the above applies.

The frame is then sealed with AES-256-GCM. Authentication costs sixteen bytes of tag, and on the slowest links that is a real fraction of the frame, so the protocol also defines a truncated eight-byte tag admitted only on transports that rate-limit authentication failures.

4.3 Measured compression

From the 36-utterance run exported on 22 September 2026, on the shipped release build:

Measure	Value	Basis
Mean frame on the wire	34.2 B	36 utterances, Hindi and English
Best observed ratio	3,310×	against 96 kB of raw PCM
Smallest frame observed	30 B	short Hindi sentence, packed
Largest frame observed	44 B	longer sentence, packed

The design figures quoted in the documentation — 2,182× unauthenticated, 1,600× authenticated, 43× against the Opus floor, and 7,385× for a template code — are computed against a fixed three-second reference. The measured numbers above are from live traffic and sit in the same range, which is the point of quoting both.

5. Radio transports and the mesh

Four transports sit behind one interface, and the application runs as many of them as the handset has. A frame that arrives twice, by two roads, is delivered once — the replay window above the link discards the second copy.

Transport	Design range	Field-measured	Note
Bluetooth Classic (RFCOMM)	10–30 m	—	Point to point, needs pairing
BLE broadcast	10–50 m	≈ 60 m	No pairing; the road that needs nothing arranged
Wi-Fi / Wi-Fi Direct	50–150 m	160–200 m	Highest throughput; group formation
Serial to LoRa or HF	2–15 km	—	For kilometre range; ~290 bps at SF12

The field-measured column is from our own range tests on the two handsets, not from a datasheet. Both radios beat their design figures outdoors. We also confirmed a relay hop working over 150 m through an

obstruction that broke the direct path — a unit out of range is reached by rebroadcast through one that is not, with the hop count set by the operator and defaulting to three.

BLE broadcast deserves a note because it turned out to shape the latency more than anything on the device. An advertisement is not a packet; it is a chance, repeated every advertising interval, and the scanner on the other handset takes only some of those chances. So the advertisement carries everything the unit has said recently and stays on the air continuously, rather than putting each frame up alone and taking it down. A receiver that hears any advertisement inside the window gets the whole buffer.

6. Security

A radio net that carries evacuation orders has an obvious attack: inject a false one. Every frame is therefore sealed with AES-256-GCM under a pre-shared key, and a frame that fails its tag is discarded before anything else looks at it.

- Authenticated encryption on every frame, including heartbeats and control traffic.
- A replay window per sender and epoch, so a captured frame cannot be re-sent later.
- Authentication-failure rate limiting, which is also what admits the truncated tag on slow links.
- Key provisioning by QR code, so joining a net does not require typing a key.

The build shipped for testing carries a development key and says so on screen in red — an unmissable banner reading UNSECURED. That is deliberate: a demonstration build that looks secure and is not is worse than one that admits it.

7. Implementation

Aspect	Detail
Language and UI	Kotlin, Jetpack Compose, coroutines
Minimum Android	8.0 (API 26)
Speech recognition	AI4Bharat IndicConformer, exported to sherpa-onnx and quantised to int8 — one ~120 MB multilingual model covering all ten languages, not one per language
Speech synthesis	Piper / VITS voices, 20–60 MB per language, fetched on demand and checksum-verified
Languages	English, Hindi, Bengali, Marathi, Telugu, Tamil, Gujarati, Kannada, Malayalam, Odia
Installer size	28.8 MB release APK, arm64-v8a, R8 minified and resource-shrunk
Codebase	8 modules · 132 source files · 79 test files · 1,627 automated tests passing
Runtime network use	None. Models are fetched once at setup; the application makes no network call while running

Aspect	Detail
Licence	Apache-2.0, with espeak-ng (GPL-3.0) and Meta MMS (CC-BY-NC) disclosed with their consequences

The 30 MB installer cap is a hard constraint and it was met with room to spare only after auditing which native libraries the application actually loads: the sherpa-onnx AAR ships four, and the Kotlin binding reaches the engine through JNI and touches two of them. Excluding the other two recovered 4.7 MB — the whole distance between 30.9 MB and the target.

8. How the numbers are measured

This section exists because most of the marks in this competition are for measurement, and because a figure without a method behind it is a claim.

8.1 Not with a stopwatch

The headline figure is a delay measured across two devices: audio leaving the speaker on the receiver, minus the microphone opening on the sender. Two Android handsets do not share a clock, so subtracting one timestamp from the other is meaningless. Human reaction time is about 250 ms against a target budget of 800 ms, so timing the demonstration by hand would make the instrument a third of the measurement.

Instead the two handsets establish a common reference by exchanging timestamped round trips over the radio link, and the receiver's timestamps are converted onto the sender's clock before any difference is taken. The offset is the median over the round trips that were not delayed in transit — a queued round trip inherits the delay asymmetrically and would poison the offset. Both figures are written to the CSV for every utterance.

8.2 Two end-to-end figures, and why

The literal figure runs from the microphone opening, so on push-to-talk it necessarily contains however long the operator held the control. A three-second sentence makes it read about 3,700 ms — and the gap is the operator's speech, not the system's delay. The comparable figure runs from the release of the control to the first sound on the other handset, which is exactly what the stage budget is a sum of. Both are exported; the second is what the application puts in its headline.

8.3 What makes a run reportable

The evaluation rules require a release build, an entry-tier handset off charge, a thirty-minute thermal soak and at least a hundred utterances before a median may be quoted. The application checks all of that itself. A run that falls short still writes its files — a measurement you are forbidden to read is not rigour — but every file opens with NOT REPORTABLE and a line naming each condition that failed. The caveat is inside the file, where a rename cannot remove it.

9. Measured results

The run below is from two retail Samsung handsets — an entry-tier SM-E066B and an SM-S947B — talking to each other over BLE broadcast on the release build, on 22 September 2026. Thirty-six utterances were timed and thirty-one completed a full round trip. The run is explicitly not reportable under the project's own rules: an 11-minute soak against a 30-minute requirement, handsets on charge, and 31 utterances against a hundred. It is shown because it is real, and labelled because it is short.

9.1 Where the time goes

Stage	Measured median	Budget	Verdict
Final decode after endpoint	306 ms	250–450 ms	Within budget
Framing, encryption, transmit	3 ms	20–60 ms	Well under budget
Link transit (BLE broadcast)	1,050 ms	—	The bottleneck
Normalisation	1 ms	< 10 ms	Within budget
First synthesis chunk	250 ms	150–250 ms	At the ceiling
Output pipeline	withheld	30–80 ms	See 9.4

The finding is unambiguous and it is not the part we expected. Everything that happens on the device is inside its budget or close to it: recognition, cryptography, normalisation and synthesis together account for roughly 560 ms. The link is where the time goes. Framing and sealing a frame costs 3 ms; getting that frame across BLE broadcast costs a second, because an advertisement waits its turn in a rotation rather than being sent on demand.

9.2 End to end

Figure	Median	p95	Samples
Release to sound	2,015 ms	7,839 ms	31
Microphone to sound	3,027 ms	9,043 ms	31
Best single utterance, quiet conditions	930 ms	—	measured separately

Against a target of 800–1,200 ms, the median of 2,015 ms misses. The best individual exchanges, with the radio quiet and the receiving handset not already speaking, came in between 930 ms and 1,330 ms — inside the budget. The difference between those two states is the whole story of the p95: this run drove both handsets simultaneously, a second apart, so the receiver was frequently still speaking the previous sentence when the next one arrived, and the slowest rows are that queue rather than a slow pipeline.

9.3 Link characterisation

- One-way BLE delay, measured by round trip: 77–113 ms once the queued samples are filtered out.

- Before that filter was added, the same pair of handsets in the same room produced one-way estimates between 113 ms and 1,242 ms, and an end-to-end figure of minus 79 milliseconds — audio arriving before the operator let go.
- That negative figure is the reason the estimator now discards round trips slower than twice the quickest in its window, and the reason a negative latency is excluded from the median and counted separately rather than averaged in. An error that flatters the result is the one nobody goes looking for.

9.4 A figure we are not quoting

The output-pipeline stage — the gap between the first synthesised chunk and the first sound — is left blank above. Every row in this run reported it as 249, 250 or 251 ms, which is not a measurement of thirty-one different utterances; it is a timeout. The instrumentation wrote a short slice of audio and waited up to 250 ms for the device's playback counter to move, and on this hardware that slice was too small to start the stream, so the wait expired every time.

The instrumentation has been corrected — the phrase is now written in slices and the counter is checked after each, so the mark is taken when sound actually starts — but the corrected figure has not yet been re-measured on both handsets. Publishing the 250 ms would have been publishing our own timeout as a result.

9.5 Footprint

Quantity	Measured	Target
Release installer	28.8 MB	< 30 MB
Acoustic model, all ten languages	~120 MB int8	fetches once
Synthesis voice, per language	20–60 MB	on demand
Frame on the wire, mean	34.2 B	—

10. Feasibility and viability

The system is built and running: speech in, radio link, speech out, on two retail handsets in aeroplane mode. The feasibility question is therefore not whether it can work but whether it holds up under the conditions it is meant for.

Risk	Why it is real	What we do about it
Recognition accuracy in noise	A flood embankment is not a quiet room, and word error rate rises sharply with noise	Alert vocabulary is biased and scored separately; below a confidence threshold the message still goes but travels marked, and the template tier does not depend on recognition at all

Risk	Why it is real	What we do about it
Thermal throttling on entry-tier hardware	Real-time factor can double after ten minutes of continuous inference	Figures are only reportable after a 30-minute soak; the build targets an entry-tier handset, never a flagship
BLE broadcast latency	Measured at ~1,050 ms median, the single largest term	Wi-Fi Direct runs alongside and is faster; frames arriving by two roads are de-duplicated. Reducing the advertising rotation is the next optimisation
Radio pairing failing on the day	Bluetooth pairing is the classic demonstration failure	BLE broadcast needs nothing arranged — no pairing, no discovery dialog
Model licensing	Two dependencies carry restrictive licences	espeak-ng (GPL-3.0) and Meta MMS (CC-BY-NC) are disclosed with their consequences; the rest is Apache-2.0 and MIT

The honest summary is that the device-side pipeline is comfortably within budget and the radio is not. That is a better position to be in than the reverse, because the radio layer has headroom we have not spent yet and the models do not.

11. Impact and benefits

11.1 Who it is for

Disaster response teams working where the towers are down. Border and forest patrols beyond coverage. Field stations on an HF or LoRa link measured in hundreds of bits per second. In each case the alternative today is either no voice at all or a handheld radio that carries one language and no text.

11.2 What changes

- Speech over links that could never carry speech — a 300 bps channel becomes usable for conversation rather than only for telemetry.
- A language barrier removed at the point it matters most: an alert sent in one Indian language is heard in another, with no translation model and no connectivity.
- Ordinary handsets instead of dedicated radios. The hardware is whatever the team already carries.
- Nothing leaves the device. No cloud dependency is not only a privacy property — it is what lets the system work at all when the infrastructure is gone.

11.3 Beyond the problem statement

The same transport carries anything that can be said in a sentence: field reports from areas with no data coverage, agricultural advisories to feature-phone-era hardware over LoRa, or accessible communication for people who cannot type. The compression argument does not depend on the use case — it depends only on the fact that meaning is smaller than sound.

12. Limitations and what remains

- The median latency is 2,015 ms against a 800–1,200 ms target. The gap is almost entirely BLE broadcast transit; the device pipeline is within budget.
- No run has yet met the reportable bar — a 30-minute soak, off charge, 100 utterances. Every figure in section 9 is therefore provisional and labelled as such in the exported files.
- The output-pipeline stage is awaiting re-measurement after the instrumentation fix described in 9.4.
- Nothing samples CPU, memory or thermal state on the live path yet, so resource.csv is written with its header and no rows rather than with invented numbers.
- Word error rate and synthesis MOS are defined and scored in the bench module but have not been run against the held-out benchmark datasets on this hardware.
- Recognition currently runs a CTC graph, which cannot take the hotword file that would let the alert lexicon bias the decoder directly; the lexicon corrects the output instead, which is measurably more dangerous and is tested as such.

We would rather state these plainly than present a system that looks finished. Every one of them is visible in the repository, in the exported CSV, or on the metrics screen of the running application.

13. References

Every link below was checked on 22 September 2026.

13.1 Models, runtimes and datasets

- **AI4Bharat IndicConformer** — the multilingual acoustic model, 22 Indian languages
<https://huggingface.co/ai4bharat/indic-conformer-600m-multilingual>
- **IndicConformerASR** — reference implementation and export scripts
<https://github.com/AI4Bharat/IndicConformerASR>
- **AI4Bharat models portal** — index of the released Indic speech and language models
<https://models.ai4bharat.org/>
- **IndicSUPERB** — the benchmark suite; Kathbath is its ASR component
<https://github.com/AI4Bharat/IndicSUPERB>
- **Kathbath** — 1,684 hours of labelled speech across 12 Indian languages
<https://huggingface.co/datasets/ai4bharat/Kathbath>
- **IndicSUPERB paper** — Javed et al., the methodology behind the benchmark
<https://arxiv.org/abs/2208.11761>
- **IndicVoices** — spontaneous rather than read speech — the realistic condition
<https://huggingface.co/datasets/ai4bharat/IndicVoices>
- **Vaani** — IISc and ART PARK, funded by Google; district-level accent coverage
<https://vaani.iisc.ac.in/>
- **Vaani on Hugging Face** — the released portion of the corpus
<https://huggingface.co/datasets/ARTPARK-IISc/Vaani>

- **FLEURS** — Google; makes figures comparable with published literature
<https://huggingface.co/datasets/google/fleurs>
- **FLEURS paper** — Conneau et al., 2022
<https://arxiv.org/abs/2205.12446>
- **Mozilla Common Voice** — supplementary validation corpus
<https://commonvoice.mozilla.org/>
- **sherpa-onnx** — the on-device inference runtime this application binds to
<https://github.com/k2-fsa/sherpa-onnx>
- **Piper** — the neural text-to-speech voices, MIT licensed
<https://github.com/rhasspy/piper>
- **Silero VAD** — voice activity detection, MIT licensed
<https://github.com/snakers4/silero-vad>
- **espeak-ng** — phonemisation data — GPL-3.0, disclosed in LICENSES.md
<https://github.com/espeak-ng/espeak-ng>

13.2 Standards and specifications

- **RFC 6716** — Definition of the Opus Audio Codec — the 6 kbps floor the compression argument is measured against
<https://www.rfc-editor.org/rfc/rfc6716.html>
- **NIST SP 800-38D** — Recommendation for Block Cipher Modes of Operation: GCM and GMAC
<https://doi.org/10.6028/NIST.SP.800-38D>
- **Bluetooth Core Specification** — LE extended advertising, on which the broadcast transport depends
<https://www.bluetooth.com/specifications/specs/core-specification/>
- **LoRaWAN Regional Parameters RP002** — spreading factor and bitrate figures for the serial transport
<https://resources.lora-alliance.org/technical-specifications/rp002-1-0-4-regional-parameters>

13.3 Competition

- **Smart India Hackathon** — the official portal, problem statements and submission rules
<https://www.sih.gov.in/>

13.4 This project

- **Source repository** — the complete implementation, tests and documentation
<https://github.com/vikranthsai310/sih2026>

The documents below are in the repository and are the normative source for the figures quoted above:

- docs/ARCHITECTURE.md — modules, threads and the full signal path.
- docs/PROTOCOL.md — frame format, script packing, template codes, AEAD, relaying.
- docs/EVALUATION.md — the definition and method behind every figure in section 9.
- docs/SECURITY.md — threat model, provisioning and audit.
- docs/MODELS.md — model choice, sizes and licence obligations.
- bench-out/latency.csv — the exported run that section 9 is computed from.